

PRODUCT REQUIREMENTS + SCRUM DELIVERY PACK

Lexora Solo-Lawyer MVP

Eight-week implementation plan for a focused legal work-to-payment platform

Plan	Decision
Target user	Independent lawyers and solo practitioners in India
Core promise	Capture every piece of legal work and turn it into a paid invoice with minimal effort.
Delivery model	8 weeks / 4 two-week sprints / 320 hours total capacity
Release target	Controlled MVP pilot with 3-5 independent lawyers
Repository	E:\LexoraV1
Integration branch	codex/solo-lawyer-mvp

PRODUCT DECISION

Position Lexora as a solo-lawyer work-to-payment product, not a legal ERP. Keep one owner per workspace, remove team approvals from the experience, and retain strict workspace isolation between customers.

1. Product strategy

Problem

Independent lawyers lose billable time across email, research, calls, drafting and court work; reconstructing that work later delays invoicing and obscures collections. Lexora should make capture and billing a single, low-friction daily habit.

North-star workflow

WORKFLOW

Login -> Client -> Matter -> Task -> Work Capture -> Review Time -> Billable -> Invoice -> Send -> Payment -> Income and Outstanding Reports

Product objectives and measures

ID	Objective	Pilot success measure
O1	Reduce lost billable work	At least 80% of pilot work entries are captured on the day performed
O2	Shorten work-to-invoice time	Median time from ready-to-bill work to sent invoice is under 10 minutes
O3	Make collections visible	Every sent invoice has an accurate outstanding balance and aging bucket
O4	Earn pilot trust	Zero cross-workspace access defects and zero duplicate-billing defects

MVP scope

- Single-owner account, onboarding and practice settings
- Clients, contacts and balances
- Matters, court details, tasks and financial summaries
- Web/manual/Gmail/research work capture with recovery
- Owner-operated work review: Draft -> Ready to Bill -> Excluded
- Rates, billables, expenses and unbilled WIP
- Professional GST invoice, PDF, delivery and revisions
- Full/partial payments, receipts, write-offs and aging
- Daily dashboard, income, WIP and outstanding reports

Explicit non-goals

- Multi-user firm administration
- RBAC and team permissions
- Partner or billing-manager approvals
- Attendance, HR and payroll
- Workforce monitoring and analytics
- Trust accounting and general ledger
- WhatsApp/SMS and meeting recording
- Automatic court feed or legal research AI
- Native mobile applications
- Deep CRM/ERP integrations during MVP

2. Scrum delivery model

The implementation uses four two-week sprints. The solo developer is Product Owner, engineer and release owner; AI is used for bounded implementation and test tasks, while product decisions, integration, security and financial verification remain human-owned.

Sprint	Goal	Branches	Plan	Buffer	Sprint review demo
S1 Weeks 1-2	Create the secure solo account foundation and reliable client/matter system.	A-D	72 h	8 h	Register a solo account, complete onboarding, create a client and matter, and prove workspace isolation.
S2 Weeks 3-4	Capture legal work from tasks, timer, manual entry, Gmail and research into one review queue.	E-H	72 h	8 h	Start work from a task, stop the timer, recover an offline capture and review one unified work entry.
S3 Weeks 5-6	Turn reviewed work and expenses into accurate, professional invoices.	I-L	72 h	8 h	Review time, apply a rate, add an expense, generate and lock a GST invoice PDF.
S4 Weeks 7-8	Send invoices, record payments, report income/outstanding balances and launch the pilot safely.	M-P	72 h	8 h	Send an invoice, record a partial payment, issue a receipt and verify dashboard/report balances.

Cadence

- Sprint planning: first morning of each sprint; confirm sprint goal, branch sequence and acceptance tests.
- Daily planning: 15 minutes; select one shippable vertical slice and state the verification command before coding.
- Daily review: 15 minutes; update tracker status, hours remaining, blockers and decision log.
- Backlog refinement: 45 minutes each Friday; split oversized items and protect the frozen non-goals.
- Sprint review: working demonstration against real API data and release criteria.
- Retrospective: record one process improvement and apply it in the next sprint.

Definition of Ready

- User outcome and route are named.
- Relevant frontend and backend files are identified.
- Data ownership and workspace-scope rules are explicit.
- Acceptance criteria are testable.

- Dependencies and migration impact are known.

Definition of Done

- Implementation is connected to real APIs; no hidden placeholder remains.
- Success, loading, empty, validation, error and retry states work.
- Workspace ownership and input validation are covered by tests.
- Frontend build and relevant backend/extension tests pass.
- The branch tracker, API mapping and known limitations are updated.
- The vertical workflow is manually verified on desktop and mobile where relevant.

Sprint exit criteria

Sprint	Exit criteria
S1	Authentication, workspace scoping, client CRUD and matter CRUD pass integration tests.
S2	Every supported capture source creates exactly one workspace-scoped work entry.
S3	Ready-to-bill work cannot be billed twice; invoice totals reconcile server-side.
S4	End-to-end regression, backups, monitoring and pilot onboarding gates pass.

3. Release gates and governance

1. Workspace A cannot read or mutate Workspace B data.
2. Every supported capture produces at most one work entry.
3. Every work entry and expense can be invoiced at most once.
4. Invoice totals, payment allocations and reports reconcile exactly.
5. Finalised invoices are locked and corrections are audited.
6. Backup restoration, error monitoring and production health checks pass.
7. All retained screens have loading, empty, error and recovery states.
8. No P0/P1 defect remains open at pilot launch.

Top delivery risks

ID	Severity	Risk	Mitigation	Due
R1	Critical	Cross-workspace data access	Mandatory workspace scoping, two-workspace tests, no caller-supplied ownership	S1
R2	Critical	Duplicate work-to-billable or billable-to-invoice conversion	Unique indexes, idempotency keys, transactional conversion and concurrency tests	S3
R3	High	Financial rounding/GST mismatch	Integer minor units, server-side totals and golden fixtures	S3
R4	High	Gmail DOM or extension instability	Retry queue, visible health state and manual-entry fallback	S2
R5	High	Solo developer scope expansion	Frozen non-goals, 8-hour sprint buffer and change-control rule	All
R6	Medium	Email/PDF provider failure	Managed provider sandbox, retry and download fallback	S4
R7	Medium	Insufficient pilot feedback	Weekly pilot interviews, event metrics and in-product feedback	S4

Change-control rule

SCOPE PROTECTION

No new product module enters the MVP after Sprint 1. A new request must replace equal or greater planned work, preserve release gates, and be recorded in the decision log.

4. Branch and work-package index

No.	Sprint	Branch	Epic	Hours	SP	Primary outcome
A	S1	codex/solo-product-shell	Foundation	12	6	Introduce solo product mode, focused navigation and shared UX states while hiding enterprise-only modules.
B	S1	codex/solo-auth-onboarding	Account	20	10	Simplify registration and login to one owner account and capture practice, tax, invoice and payment defaults during onboarding.
C	S1	codex/workspace-isolation	Security	20	10	Enforce workspace ownership across every retained resource and migrate legacy firm records safely.
D	S1	codex/solo-clients-matters	Practice records	20	10	Complete client and matter records for a solo practice, including contacts, GST, billing terms, court details and financial summaries.
E	S2	codex/solo-tasks-daily-work	Daily work	20	10	Simplify tasks for one lawyer and allow any task to launch a pre-filled work session.
F	S2	codex/solo-work-meter-manual	Work capture	24	12	Harden web timer and manual entry with timezone, idle, rounding, recovery and duplicate protection.
G	S2	codex/gmail-research-capture	Work capture	16	8	Preserve differentiated Gmail and research capture while simplifying setup for one owner.
H	S2	codex/unified-work-review	Time review	12	6	Replace approval queues with one owner-operated work review experience.
I	S3	codex/solo-rates-billables	Billing	16	8	Resolve default, client and matter rates and create billables exactly once from reviewed work.
J	S3	codex/expenses-disbursements	Billing	18	9	Add reimbursable legal expenses and receipt attachments to matter billing.
K	S3	codex/invoice-core-controls	Invoices	22	11	Create a single canonical invoice flow from ready billables with numbering, GST, discounts, locking and revisions.
L	S3	codex/invoice-pdf-branding	Invoices	16	8	Produce one polished professional invoice PDF with solo-practice branding and payment instructions.
M	S4	codex/invoice-email-delivery	Collections	12	6	Send finalised invoices through managed email with delivery history and retry states.
N	S4	codex/solo-payments-receipts	Payments	16	8	Support full/partial payments, reconciliation, receipts, write-offs and accurate outstanding balances.
O	S4	codex/solo-dashboard-reports	Insights	18	9	Give the lawyer an actionable daily dashboard plus income, WIP, invoice and outstanding reports.
P	S4	codex/solo-release-hardening	Release	26	13	Prepare a safe controlled pilot with monitoring, backups, accessibility, mobile polish and end-to-end regression.

A - codex/solo-product-shell

S1 | Foundation | 12 planned hours | 6 story points

OUTCOME

Introduce solo product mode, focused navigation and shared UX states while hiding enterprise-only modules.

Delivery field	Branch requirement
Branch	codex/solo-product-shell
Screens / states	Focused app shell; solo navigation; fallback states; not-found and offline states
Backend / API	PRODUCT_MODE configuration; route exposure map; no schema change
Acceptance criteria	Only focused MVP routes appear; excluded modules cannot be reached through navigation; loading, empty, error and retry states are reusable.
Verification	Frontend build; route smoke test; desktop/mobile navigation check
Commit	feat(solo): add focused product shell

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/solo-product-shell` based on `codex/solo-lawyer-mvp`. The required outcome is: Introduce solo product mode, focused navigation and shared UX states while hiding enterprise-only modules. Implement these screens and states: Focused app shell; solo navigation; fallback states; not-found and offline states. Backend/API scope: PRODUCT_MODE configuration; route exposure map; no schema change. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: Only focused MVP routes appear; excluded modules cannot be reached through navigation; loading, empty, error and retry states are reusable.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.

- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.
- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(solo): add focused product shell

B - codex/solo-auth-onboarding

S1 | Account | 20 planned hours | 10 story points

OUTCOME

Simplify registration and login to one owner account and capture practice, tax, invoice and payment defaults during onboarding.

Delivery field	Branch requirement
Branch	codex/solo-auth-onboarding
Screens / states	Login; register; forgot/reset password; onboarding; owner profile; practice settings
Backend / API	Auth current-user routes; workspace creation; owner profile/settings; session revocation
Acceptance criteria	A lawyer registers without choosing a role, receives one workspace and can complete/edit practice settings.
Verification	Auth route tests; reset flow; onboarding persistence; session-expiry test
Commit	feat(auth): add solo owner onboarding

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/solo-auth-onboarding` based on `codex/solo-lawyer-mvp`. The required outcome is: Simplify registration and login to one owner account and capture practice, tax, invoice and payment defaults during onboarding. Implement these screens and states: Login; register; forgot/reset password; onboarding; owner profile; practice settings. Backend/API scope: Auth current-user routes; workspace creation; owner profile/settings; session revocation. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: A lawyer registers without choosing a role, receives one workspace and can complete/edit practice settings.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.

- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(auth): add solo owner onboarding

C - codex/workspace-isolation

S1 | Security | 20 planned hours | 10 story points

OUTCOME

Enforce workspace ownership across every retained resource and migrate legacy firm records safely.

Delivery field	Branch requirement
Branch	codex/workspace-isolation
Screens / states	No new primary screen; access-denied and migration-safe error states
Backend / API	workspaceId/firmId migration; request workspace context; scoped queries; ownership validators
Acceptance criteria	Workspace A cannot read or mutate Workspace B records; ownership fields cannot be supplied by clients.
Verification	Two-workspace security suite across clients, matters, tasks, work, invoices and payments
Commit	security: enforce solo workspace isolation

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/workspace-isolation` based on `codex/solo-lawyer-mvp`. The required outcome is: Enforce workspace ownership across every retained resource and migrate legacy firm records safely. Implement these screens and states: No new primary screen; access-denied and migration-safe error states. Backend/API scope: workspaceId/firmId migration; request workspace context; scoped queries; ownership validators. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: Workspace A cannot read or mutate Workspace B records; ownership fields cannot be supplied by clients.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.

- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: security: enforce solo workspace isolation

D - codex/solo-clients-matters

S1 | Practice records | 20 planned hours | 10 story points

OUTCOME

Complete client and matter records for a solo practice, including contacts, GST, billing terms, court details and financial summaries.

Delivery field	Branch requirement
Branch	codex/solo-clients-matters
Screens / states	Client list/form/detail; matter list/form/detail; client balance; matter financial summary
Backend / API	Client contacts/notes; matter numbering; matter billing method; scoped summaries
Acceptance criteria	Client and matter CRUD works with validation, archive/reopen, search and workspace ownership.
Verification	CRUD integration tests; duplicate warning; archive/reopen; summary reconciliation
Commit	feat(practice): complete solo clients and matters

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/solo-clients-matters` based on `codex/solo-lawyer-mvp`. The required outcome is: Complete client and matter records for a solo practice, including contacts, GST, billing terms, court details and financial summaries. Implement these screens and states: Client list/form/detail; matter list/form/detail; client balance; matter financial summary. Backend/API scope: Client contacts/notes; matter numbering; matter billing method; scoped summaries. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: Client and matter CRUD works with validation, archive/reopen, search and workspace ownership.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.

- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(practice): complete solo clients and matters

E - codex/solo-tasks-daily-work

S2 | Daily work | 20 planned hours | 10 story points

OUTCOME

Simplify tasks for one lawyer and allow any task to launch a pre-filled work session.

Delivery field	Branch requirement
Branch	codex/solo-tasks-daily-work
Screens / states	Task list; task form; task detail; My Work Today
Backend / API	Task CRUD; due/priority/checklist; meter context handoff
Acceptance criteria	The owner can create, complete and filter tasks and launch the timer with client/matter/task context.
Verification	Task route tests; overdue filter; checklist persistence; meter handoff
Commit	feat(tasks): add solo daily-work flow

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/solo-tasks-daily-work` based on `codex/solo-lawyer-mvp`. The required outcome is: Simplify tasks for one lawyer and allow any task to launch a pre-filled work session. Implement these screens and states: Task list; task form; task detail; My Work Today. Backend/API scope: Task CRUD; due/priority/checklist; meter context handoff. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: The owner can create, complete and filter tasks and launch the timer with client/matter/task context.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.
- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(tasks): add solo daily-work flow

F - codex/solo-work-meter-manual

S2 | Work capture | 24 planned hours | 12 story points

OUTCOME

Harden web timer and manual entry with timezone, idle, rounding, recovery and duplicate protection.

Delivery field	Branch requirement
Branch	codex/solo-work-meter-manual
Screens / states	Work Meter; current session; session history; manual time form; recovery state
Backend / API	Work-session lifecycle; firm timezone; idempotency; recovery; work-entry projection
Acceptance criteria	Start/pause/resume/stop and manual entry each create one accurate draft work entry with recoverable failure states.
Verification	Lifecycle tests; timezone boundaries; idle calculation; duplicate request; crash recovery
Commit	feat(work): harden timer and manual capture

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/solo-work-meter-manual` based on `codex/solo-lawyer-mvp`. The required outcome is: Harden web timer and manual entry with timezone, idle, rounding, recovery and duplicate protection. Implement these screens and states: Work Meter; current session; session history; manual time form; recovery state. Backend/API scope: Work-session lifecycle; firm timezone; idempotency; recovery; work-entry projection. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: Start/pause/resume/stop and manual entry each create one accurate draft work entry with recoverable failure states.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.

- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(work): harden timer and manual capture

G - codex/gmail-research-capture

S2 | Work capture | 16 planned hours | 8 story points

OUTCOME

Preserve differentiated Gmail and research capture while simplifying setup for one owner.

Delivery field	Branch requirement
Branch	codex/gmail-research-capture
Screens / states	Extension setup/status; Gmail review; research review; capture mapping
Backend / API	Extension token; email entries; research capture; mapping and idempotent conversion
Acceptance criteria	Offline captures retry safely; duplicate source references do not duplicate work; mapping updates the unified queue.
Verification	Extension unit tests; queue retry; duplicate capture; workspace mapping
Commit	feat(capture): adapt Gmail and research for solo use

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/gmail-research-capture` based on `codex/solo-lawyer-mvp`. The required outcome is: Preserve differentiated Gmail and research capture while simplifying setup for one owner. Implement these screens and states: Extension setup/status; Gmail review; research review; capture mapping. Backend/API scope: Extension token; email entries; research capture; mapping and idempotent conversion. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: Offline captures retry safely; duplicate source references do not duplicate work; mapping updates the unified queue.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.
- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(capture): adapt Gmail and research for solo use

H - codex/unified-work-review

S2 | Time review | 12 planned hours | 6 story points

OUTCOME

Replace approval queues with one owner-operated work review experience.

Delivery field	Branch requirement
Branch	codex/unified-work-review
Screens / states	Work Entries; entry edit drawer; excluded-work filter; ready-to-bill queue
Backend / API	Status adapter: draft/ready_to_bill/excluded/billed; audit history; batch actions
Acceptance criteria	The owner can correct duration/narrative/matter/rate, exclude work or mark it ready to bill without approval roles.
Verification	State-transition tests; edit-lock rules; batch selection; legacy status migration
Commit	feat(work): add unified solo review queue

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/unified-work-review` based on `codex/solo-lawyer-mvp`. The required outcome is: Replace approval queues with one owner-operated work review experience. Implement these screens and states: Work Entries; entry edit drawer; excluded-work filter; ready-to-bill queue. Backend/API scope: Status adapter: draft/ready_to_bill/excluded/billed; audit history; batch actions. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: The owner can correct duration/narrative/matter/rate, exclude work or mark it ready to bill without approval roles.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.
- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(work): add unified solo review queue

I - codex/solo-rates-billables

S3 | Billing | 16 planned hours | 8 story points

OUTCOME

Resolve default, client and matter rates and create billables exactly once from reviewed work.

Delivery field	Branch requirement
Branch	codex/solo-rates-billables
Screens / states	Rate settings; billing workbench; unbilled WIP
Backend / API	Effective rates; rate precedence; amount snapshot; idempotent billable conversion
Acceptance criteria	Correct rate precedence applies; write-up/write-down needs a reason; one work entry produces no more than one billable.
Verification	Rate matrix tests; overlap validation; duplicate conversion; WIP totals
Commit	feat(billing): simplify rates and billables

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/solo-rates-billables` based on `codex/solo-lawyer-mvp`. The required outcome is: Resolve default, client and matter rates and create billables exactly once from reviewed work. Implement these screens and states: Rate settings; billing workbench; unbilled WIP. Backend/API scope: Effective rates; rate precedence; amount snapshot; idempotent billable conversion. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: Correct rate precedence applies; write-up/write-down needs a reason; one work entry produces no more than one billable.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.
- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(billing): simplify rates and billables

J - codex/expenses-disbursements

S3 | Billing | 18 planned hours | 9 story points

OUTCOME

Add reimbursable legal expenses and receipt attachments to matter billing.

Delivery field	Branch requirement
Branch	codex/expenses-disbursements
Screens / states	Expense list; expense form; receipt preview; matter expenses
Backend / API	Expense model/routes; categories; tax; receipt metadata; billable conversion
Acceptance criteria	An expense can be created, attached to a matter, marked reimbursable and included once in an invoice.
Verification	Expense CRUD; receipt authorization; tax calculation; duplicate billing
Commit	feat(expenses): add matter disbursements

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/expenses-disbursements` based on `codex/solo-lawyer-mvp`. The required outcome is: Add reimbursable legal expenses and receipt attachments to matter billing. Implement these screens and states: Expense list; expense form; receipt preview; matter expenses. Backend/API scope: Expense model/routes; categories; tax; receipt metadata; billable conversion. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: An expense can be created, attached to a matter, marked reimbursable and included once in an invoice.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.
- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(expenses): add matter disbursements

K - codex/invoice-core-controls

S3 | Invoices | 22 planned hours | 11 story points

OUTCOME

Create a single canonical invoice flow from ready billables with numbering, GST, discounts, locking and revisions.

Delivery field	Branch requirement
Branch	codex/invoice-core-controls
Screens / states	Invoice list; builder; preview; detail; void/revision dialog
Backend / API	Invoice sequence; server totals; minor-unit money; finalise/lock; revision/void
Acceptance criteria	Invoice totals reconcile; finalised invoices are immutable; work/expenses cannot be invoiced twice.
Verification	Concurrency test; GST/rounding fixtures; lock/void/revision; duplicate invoice test
Commit	feat(invoices): add commercial invoice controls

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/invoice-core-controls` based on `codex/solo-lawyer-mvp`. The required outcome is: Create a single canonical invoice flow from ready billables with numbering, GST, discounts, locking and revisions. Implement these screens and states: Invoice list; builder; preview; detail; void/revision dialog. Backend/API scope: Invoice sequence; server totals; minor-unit money; finalise/lock; revision/void. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: Invoice totals reconcile; finalised invoices are immutable; work/expenses cannot be invoiced twice.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.

- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(invoices): add commercial invoice controls

L - codex/invoice-pdf-branding

S3 | Invoices | 16 planned hours | 8 story points

OUTCOME

Produce one polished professional invoice PDF with solo-practice branding and payment instructions.

Delivery field	Branch requirement
Branch	codex/invoice-pdf-branding
Screens / states	Invoice PDF preview; branding settings; print/download states
Backend / API	PDF/HTML generation; logo metadata; bank/UPI instructions
Acceptance criteria	PDF matches invoice totals, includes required identity/tax/payment fields and renders consistently.
Verification	Golden PDF fixture; long-line wrapping; GST/no-GST; download authorization
Commit	feat(invoices): add branded invoice PDF

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/invoice-pdf-branding` based on `codex/solo-lawyer-mvp`. The required outcome is: Produce one polished professional invoice PDF with solo-practice branding and payment instructions. Implement these screens and states: Invoice PDF preview; branding settings; print/download states. Backend/API scope: PDF/HTML generation; logo metadata; bank/UPI instructions. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: PDF matches invoice totals, includes required identity/tax/payment fields and renders consistently.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.

- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(invoices): add branded invoice PDF

M - codex/invoice-email-delivery

S4 | Collections | 12 planned hours | 6 story points

OUTCOME

Send finalised invoices through managed email with delivery history and retry states.

Delivery field	Branch requirement
Branch	codex/invoice-email-delivery
Screens / states	Send invoice dialog; delivery timeline; resend/failure state
Backend / API	Email provider adapter; delivery events; audit trail; retry
Acceptance criteria	A finalised invoice sends to validated recipients and records sent/failed/delivered events without duplicate sends.
Verification	Provider sandbox; idempotency; failure retry; audit event
Commit	feat(invoices): add tracked email delivery

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/invoice-email-delivery` based on `codex/solo-lawyer-mvp`. The required outcome is: Send finalised invoices through managed email with delivery history and retry states. Implement these screens and states: Send invoice dialog; delivery timeline; resend/failure state. Backend/API scope: Email provider adapter; delivery events; audit trail; retry. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: A finalised invoice sends to validated recipients and records sent/failed/delivered events without duplicate sends.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.
- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(invoices): add tracked email delivery

N - codex/solo-payments-receipts

S4 | Payments | 16 planned hours | 8 story points

OUTCOME

Support full/partial payments, reconciliation, receipts, write-offs and accurate outstanding balances.

Delivery field	Branch requirement
Branch	codex/solo-payments-receipts
Screens / states	Payment list/form; invoice payment panel; receipt; write-off dialog; aging view
Backend / API	Payment allocation; invoice status recompute; receipt generation; write-off audit
Acceptance criteria	Partial/full payments update outstanding balances; void invoices reject payments; receipts match cleared payments.
Verification	Allocation tests; overpayment rule; reconciliation; write-off; receipt totals
Commit	feat(payments): complete solo collections flow

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/solo-payments-receipts` based on `codex/solo-lawyer-mvp`. The required outcome is: Support full/partial payments, reconciliation, receipts, write-offs and accurate outstanding balances. Implement these screens and states: Payment list/form; invoice payment panel; receipt; write-off dialog; aging view. Backend/API scope: Payment allocation; invoice status recompute; receipt generation; write-off audit. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: Partial/full payments update outstanding balances; void invoices reject payments; receipts match cleared payments.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.

- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(payments): complete solo collections flow

O - codex/solo-dashboard-reports

S4 | Insights | 18 planned hours | 9 story points

OUTCOME

Give the lawyer an actionable daily dashboard plus income, WIP, invoice and outstanding reports.

Delivery field	Branch requirement
Branch	codex/solo-dashboard-reports
Screens / states	Dashboard; income report; outstanding/aging; time activity; CSV export
Backend / API	Dashboard summary; bounded report queries; CSV exports; data-quality warnings
Acceptance criteria	Dashboard values reconcile with work, invoice and payment sources for the same filters and timezone.
Verification	Report reconciliation fixtures; empty states; CSV output; filter boundaries
Commit	feat(reports): add solo income and outstanding insights

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/solo-dashboard-reports` based on `codex/solo-lawyer-mvp`. The required outcome is: Give the lawyer an actionable daily dashboard plus income, WIP, invoice and outstanding reports. Implement these screens and states: Dashboard; income report; outstanding/aging; time activity; CSV export. Backend/API scope: Dashboard summary; bounded report queries; CSV exports; data-quality warnings. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: Dashboard values reconcile with work, invoice and payment sources for the same filters and timezone.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.
- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: feat(reports): add solo income and outstanding insights

P - codex/solo-release-hardening

S4 | Release | 26 planned hours | 13 story points

OUTCOME

Prepare a safe controlled pilot with monitoring, backups, accessibility, mobile polish and end-to-end regression.

Delivery field	Branch requirement
Branch	codex/solo-release-hardening
Screens / states	Onboarding checklist; support/feedback; production error states; responsive polish
Backend / API	Health/monitoring; backup/restore; rate limits; indexes; audit review; seed/pilot tools
Acceptance criteria	All release gates pass; no P0/P1 defects; backup restore works; 3-5 pilot lawyers can onboard with documented limits.
Verification	Full test/build/E2E; restore drill; accessibility; mobile; load smoke; pilot checklist
Commit	test(release): harden solo lawyer MVP

Codex implementation prompt

You are a senior full-stack engineer implementing Lexora Solo, a focused legal work-to-payment product for independent lawyers. Work on branch `codex/solo-release-hardening` based on `codex/solo-lawyer-mvp`. The required outcome is: Prepare a safe controlled pilot with monitoring, backups, accessibility, mobile polish and end-to-end regression. Implement these screens and states: Onboarding checklist; support/feedback; production error states; responsive polish. Backend/API scope: Health/monitoring; backup/restore; rate limits; indexes; audit review; seed/pilot tools. Inspect existing routes, controllers, models, validators, tests and frontend API clients before editing. Reuse existing components and preserve legacy data compatibility. All retained records must be scoped to the authenticated workspace; never trust ownership fields from the client. Use real API data and honest recovery states. The work is complete only when: All release gates pass; no P0/P1 defects; backup restore works; 3-5 pilot lawyers can onboard with documented limits.

Implementation guardrails

- Make additive schema changes first; migrate and verify before removing legacy fields or states.
- Prefer one canonical workflow over parallel conversion paths.
- Calculate financial totals server-side and preserve idempotency for every conversion.
- Add tests with the implementation, including a failure or boundary case.
- Do not reintroduce multi-user approvals, RBAC, HR/payroll or enterprise administration.

- Do not commit secrets, generated build output or unrelated user changes.

Required completion response

1. Implementation summary
2. Screens/routes completed
3. API/schema changes
4. Migration or compatibility notes
5. Tests and exact verification commands
6. Known limitations
7. Suggested commit: test(release): harden solo lawyer MVP

5. Screen inventory

Module	Screen	Route	Action	Sprint	Branch
Account	Login	/login	Keep / simplify	S1	B
Account	Register	/register	Revise	S1	B
Account	Password recovery	/forgot-password	Complete	S1	B
Account	Solo onboarding	/onboarding	New	S1	B
Account	Owner profile/settings	/app/settings	Revise	S1	B
Dashboard	Solo dashboard	/app/dashboard	Revise	S4	O
Clients	Client list	/app/clients	Keep / harden	S1	D
Clients	Client form	/app/clients/new	Keep / harden	S1	D
Clients	Client detail & balance	/app/clients/:id	Revise	S1	D
Matters	Matter list	/app/matters	Keep / harden	S1	D
Matters	Matter form	/app/matters/new	Revise	S1	D
Matters	Matter workspace	/app/matters/:id	Revise	S1	D
Tasks	Task list / today	/app/tasks	Simplify	S2	E
Tasks	Task form/detail	/app/tasks/:id	Keep / harden	S2	E
Work	Work Meter	/app/work-meter	Keep / harden	S2	F
Work	Manual time entry	/app/work-entries/new	Revise	S2	F
Work	Unified Work Entries	/app/work-entries	New/compose	S2	H
Work	Entry edit/review	/app/work-entries/:id	New/compose	S2	H
Capture	Extension setup/status	/app/capture/setup	Simplify	S2	G
Capture	Gmail/research review	/app/captured-work	Compose	S2	G
Billing	Rates settings	/app/settings/rates	Revise	S3	I
Billing	Billing workbench / WIP	/app/billing	Revise	S3	I
Expenses	Expense list	/app/expenses	New	S3	J
Expenses	Expense form	/app/expenses/new	New	S3	J
Invoices	Invoice list	/app/invoices	Keep / harden	S3	K
Invoices	Invoice builder	/app/invoices/new	Revise	S3	K
Invoices	Invoice detail	/app/invoices/:id	Keep / harden	S3	K
Invoices	PDF preview	/app/invoices/:id/preview	Complete	S3	L
Invoices	Send/delivery history	/app/invoices/:id/send	Complete	S4	M

Module	Screen	Route	Action	Sprint	Branch
Payments	Payments and allocation	/app/payments	Keep / harden	S4	N
Payments	Receipt	/app/payments/:id/receipt	New	S4	N
Reports	Income and outstanding	/app/reports	Revise	S4	O

6. Pilot launch playbook

Pilot cohort

Recruit 3-5 independent lawyers who currently track work manually or in spreadsheets. Use separate workspaces, realistic client/matter data and at least one invoice cycle per participant.

Pilot week measures

- Activation: account created, practice settings completed, first client and matter created.
- Capture: first timer/manual/Gmail entry recorded and reviewed.
- Billing: first ready-to-bill item and invoice generated.
- Collection: payment recorded or outstanding invoice correctly aged.
- Quality: capture failures, duplicate prevention events, invoice corrections and support requests.

Go / no-go decision

GO

Proceed beyond controlled pilot only when all release gates pass, at least three lawyers complete the end-to-end flow, and no financial or isolation defect remains unresolved.

7. Future integration boundary

After the solo workflow is stable, expose bounded integration contracts instead of expanding into a full ERP. Clients and matters may sync from CRM/practice systems; invoices and payments may sync with accounting systems.

Prioritised options: workspace-scoped REST API; outbound webhooks for work-ready, invoice-sent and payment-cleared events; CSV migration/accountant handoff; Zoho CRM/Books, QuickBooks or Xero adapters; Google/Microsoft calendar connections; and practice-management platform connectors.